

Подготовка окружения: Node.js, npm и Vite

```
● ● ● >_ bash - full-toolchain
```

```
# 1. Проверка системного рантайма
```

```
$ node -v && npm -v # → Node v20.x LTS | npm v10.x
```

```
# 2. Инициализация через нативный шаблон Vite
```

```
$ npm create vite@latest my-app -- --template react
```

```
# 3. Мгновенный запуск No-Bundle сервера разработки
```

```
$ cd my-app && npm install && npm run dev
```

Архитектура Node.js и выполнение JS вне браузера

🔧 1. Внутреннее устройство движка V8 и библиотеки libuv

Исторически язык JavaScript проектировался как строго изолированный сценарий, функционирующий внутри браузерной «песочницы» (Sandbox). Браузерный JS намеренно лишен прямого доступа к аппаратным ресурсам: он не имеет прав считывать файлы с произвольных секторов диска, открывать низкоуровневые TCP-сокеты или управлять системными процессами операционной системы.

В 2009 году Райан Дал объединил производительный C++ движок **Google V8** с системной библиотекой **libuv** и платформенными привязками (Bindings), создав среду **Node.js**. В этой архитектуре JavaScript получил полноценный статус системного языка программирования общего назначения.

```
| Node.js Runtime Environment |
|
| Google V8 Engine (C++) | libuv Platform (C++) | | - Парсинг
исходного кода в AST | - Event Loop (Цикл событий) | | - Ignition
(Интерпретатор) | - Thread Pool (4 фоновых потока) | | - TurboFan
(JIT-оптимизация) | - Cross-platform Async I/O | | - Generational
Garbage Collector | - Non-blocking network sockets |
|
| Node.js Core API Bindings (fs, path, http, crypto, os, child) |
|
| Операционная система (POSIX / Win32) |
```



Ignition Interpreter

Парсит исходный текст программы в AST и генерирует компактный оптимизированный байт-код для моментального старта.



TurboFan JIT Compiler

Анализирует профили типов «горячих» функций и на лету компилирует их в прямой бинарный машинный код процессора.



Двухуровневый GC

Очищает оперативную память через разделение на молодое поколение (Scavenge) и взрослое поколение (Mark-Sweep).



2. Event Loop и модель неблокирующего ввода-вывода (Non-blocking I/O)

В отличие от классических многопоточных серверов (Apache HTTP, Tomcat), создающих тяжелый поток ОС на каждое входящее соединение, Node.js оперирует **однопоточным циклом событий (Single-Threaded Event Loop)**, делегируя блокирующие операции пулу рабочих потоков libuv.

Исполнение задач в цикле событий разделено на строгие последовательные фазы:

- **Timers Phase:** Исполнение таймеров `setTimeout()` и `setInterval()`, срок ожидания которых истек.
- **Pending Callbacks:** Обработка системных ошибок ввода-вывода (например, сбои DNS или сокетов TCP).
- **Poll Phase:** Извлечение новых событий ввода-вывода (чтение файлов с диска, сетевые входящие запросы).
- **Check Phase:** Моментальный вызов колбэков, зарегистрированных через `setImmediate()`.
- **Close Callbacks:** Обработка событий закрытия дескрипторов соединений (`socket.on('close')`).

❗ Приоритет микротасок: `process.nextTick()` и `Promise`

Очереди микротасок (`process.nextTickQueue` и `microtaskQueue` промисов) обладают абсолютным приоритетом над всеми фазами Event Loop: они опустошаются до конца сразу после завершения текущей JS-инструкции.

3. Роль Node.js в современном фронтенд-инструментарии

При разработке Single Page Applications на React среда Node.js **не исполняет код в браузере конечного пользователя**. Ее назначение — служить платформой автоматизации инструментального конвейера (Build Toolchain):

- Транспилиция диалектов JSX и TypeScript в валидный стандартный JavaScript (ES6+).
- Препроцессинг, автопрефиксинг и изоляция CSS-модулей (PostCSS, Tailwind CSS).
- Глубокий Tree-shaking (удаление мертвого неиспользуемого кода) и генерация Source Maps.
- Хостинг локального сервера разработки с поддержкой WebSocket-соединения для моментального обновления кода.

Пакетный менеджер npm и экосистема зависимостей

1. Архитектура реестра npm

npm (Node Package Manager) — глобальная инфраструктура управления открытым кодом, состоящая из трех частей:

1. **Общедоступный реестр (Registry):** Облачная база данных, хранящая метаданные и сжатые `.tar.gz` архивы миллионов модулей.
2. **Клиент командной строки (CLI):** Консольный инструмент разработчика, выполняющий разрешение графов версий и скачивание пакетов.
3. **Файл манифеста `package.json`:** Паспорт модуля, декларирующий его имя, версию, скрипты и список зависимостей.

2. Спецификация семантического версионирования (SemVer 2.0.0)

Версия любого пакета в экосистеме обязана строго следовать формуле трех чисел: **MAJOR.MINOR.PATCH** (например, **18.3.1**):

MAJOR . MINOR . PATCH (Пример: 18.3.1) | | | | | PATCH:
Исправление багов (Сохранение публичного API) | MINOR:
Добавление новой функциональности (Обратно совместимо)
MAJOR: Breaking Changes (Нарушение совместимости API)

Синтаксис	Диапазон версий	Поведение при обновлении
<code>^1.2.3</code> (Каретка)	<code>≥ 1.2.3 < 2.0.0</code>	Разрешает минорные релизы и патчи (стандарт npm).
<code>~1.2.3</code> (Тильда)	<code>≥ 1.2.3 < 1.3.0</code>	Разрешает исключительно патчи багов внутри текущего минора.
<code>1.2.3</code> (Фиксация)	<code>≡ 1.2.3</code>	Запрещает любые автоматические изменения версии.

3. Разделение зависимостей: dependencies vs devDependencies

Точное разделение зависимостей в манифесте критично для предотвращения попадания лишнего инструментального кода в продакшен-бандл пользователя:



dependencies

Библиотеки, необходимые для непосредственного исполнения логики в браузере клиента (например, `react`, `react-dom`). Устанавливаются стандартной командой `npm i <pkg>`.



devDependencies (-D)

Инструменты сборщика, компиляторы и транспиляторы, нужные только инженеру на ПК во время разработки (`vite`, `eslint`, `typescript`). Устанавливаются с флагом `npm i -D <pkg>`.

4. Механика утилиты npx

Утилита **npx (Node Package Execute)** исключает проблему загрязнения глобального пространства операционной системы (`npm install -g`). Она проверяет наличие бинарного файла в локальном каталоге `node_modules/.bin` ; если файл отсутствует, npx скачивает пакет во временный кэш, выполняет команду и мгновенно очищает диск.

Дерево зависимостей, алгоритм Flattening и детерминизм

🔗 1. Эволюция node_modules и алгоритм уплотнения (Flattening)

В ранних версиях npm зависимости устанавливались вложенно и рекурсивно. Если модуль A требовал модуль B, а тот — модуль C, создавался глубокий путь `node_modules/A/node_modules/B/node_modules/C`. Это вызывало дублирование одинаковых пакетов и приводило к ошибке превышения длины пути в Windows (ограничение `MAX_PATH` = 260 символов).

Начиная с npm v3+, был внедрен **алгоритм выравнивания (Flattening Algorithm)**. Менеджер пакетов поднимает максимальное количество транзитивных библиотек на верхний уровень корневого каталога

`node_modules/`:

```
ВЛОЖЕННАЯ СТРУКТУРА (npm v2): ПЛОСКАЯ СТРУКТУРА (Modern npm v3+):  
node_modules/ node_modules/ ├── package-A/ ├── package-A/ | └─  
node_modules/ ├── package-B/ | └─ lodash@4.0.0 ├── lodash@4.0.0  
(Общий хост) └─ package-B/ └─ package-C/ └─ node_modules/ └─  
node_modules/ └─ lodash@4.0.0 └─ lodash@3.0.0 (Конфликт версий)
```

2. Детерминизм и роль файла `package-lock.json`

Файл `package-lock.json` представляет собой строгий криптографический слепок графа зависимостей. Он блокирует точный URL-адрес скачивания и хэш целостности **integrity (SHA-512)** каждого установленного пакета, гарантируя, что билд на удаленном сервере хостинга соберется на 100% идентично вашему компьютеру.

Критерий сравнения	Команда <code>npm install</code>	Команда <code>npm ci</code> (Clean Install)
Основной источник	Читает <code>package.json</code>	Читает строго <code>package-lock.json</code>
Изменение lock-файла	Имеет право обновить lock-файл	Никогда не меняет lock (выдает ошибку при несовпадении)
Состояние <code>node_modules</code>	Дописывает поверх существующих файлов	Полностью удаляет <code>node_modules</code> перед установкой
Область применения	Локальная разработка на компьютере	CI/CD пайплайны и продакшен-серверы

Эволюция бандлеров и No-Bundle архитектура Vite

⌚ 1. Ограничения классического бандлинга (Webpack / CRA)

Классические сборщики базировались на парадигме **полного предварительного бандлинга**. Перед тем как сервер разработки отдаст первый байт в браузер, сборщик обязан полностью проанализировать граф зависимостей, прочитать с диска тысячи модулей и склеить их в тяжелый виртуальный файл в оперативной памяти:

```
КЛАССИЧЕСКИЙ BUNDLER DEV SERVER (Webpack): [Старт dev-сервера] —>  
[Парсинг 3000 файлов] —> [Сборка бандла в RAM] —> [Сервер готов  
через 40 сек] | (Любая правка = пересборка)
```

2. Нативные модули браузера (Native ESM) и архитектура Vite

Современные браузеры нативно поддерживают стандарт модулей `<script type="module">`. Vite исключил этап предварительной сборки: dev-сервер стартует за **50 миллисекунд**, отдавая статические файлы напрямую. Браузер сам запрашивает нужные модули через HTTP-запросы по мере их вызова, а Vite транпилирует только запрашиваемый файл «на лету».

Движок	Технологический стек	Область ответственности	Главное преимущество
esbuild	Компилируемый язык Go	Pre-bundling зависимостей, транпиляция TS/JSX	В 10–100 раз быстрее инструментов на Node.js благодаря параллелизму ядер CPU.
Rollup	JavaScript	Финальная оптимизация продакшен-бандла (<code>npm run build</code>)	Глубокий Tree-shaking, разделение кода на чанки и стабильная экосистема плагинов.

3. Механизм предварительной сборки (Pre-bundling)

При первом запуске проекта Vite компилирует все библиотеки из `node_modules` в оптимизированный кэш `node_modules/.vite` с помощью `esbuild`. Это решает две фундаментальные задачи:

- **Преобразование CommonJS в ESM:** Конвертирует устаревший синтаксис `require/module.exports` в нативный `import/export`.
- **Устранение проблемы каскадных сетевых запросов (Waterfall):** Объединяет библиотеки, состоящие из сотен микрофайлов (например, `lodash-es` содержит более 600 файлов), в один компактный модуль, защищая сетевой стек браузера от перегрузки.

Механика Hot Module Replacement (HMR) и Dev-процесс

1. Архитектура HMR over WebSocket

В эпоху Live Reload любое сохранение файла вызывало полную перезагрузку окна браузера

(`window.location.reload()`) с потерей введенных в формы данных и сбросом фокуса. Протокол **HMR (Hot Module Replacement)** производит точечную замену модуля прямо в оперативной памяти запущенного приложения:

```
1. [Разработчик нажимает Ctrl+S в Button.jsx] | 2. [Файловый вотчер Chokidar в Node.js фиксирует событие изменения файла] | 3. [Vite Server выполняет обход Module Graph и определяет границу обновления (HMR Boundary)] | 4. [Отправка управляющего JSON-сообщения по протоколу WebSocket в браузер]: { "type": "update", "updates": [{ "path": "/src/Button.jsx", "timestamp": 1725123456 }] } | 5. [HMR Runtime в браузере выполняет dynamic import с временной меткой]: import('/src/Button.jsx?t=1725123456').then(newModule => { // React Fast Refresh бесшовно подменяет функцию компонента в памяти });
```

2. Инвалидация модульного графа (HMR Bubbling)

Если измененный файл не может быть заменен локально (например, модифицирован глобальный вспомогательный модуль без сохранения состояния), сигнал обновления поднимается вверх по дереву импортов (Bubble Up) до ближайшего родительского модуля, способного принять обновление. В случае недостижимости безопасной границы Vite выполняет резервный сброс (fallback) в виде полной перезагрузки страницы.